

d) Scraping completo de comentarios (testimonios) con BeautifulSoup y paginación

Objetivo: descargar **todos** los testimonios generales de clientes (/testimonios) y guardarlos en data/comentarios.csv .

sitemap-testimonios.xml solo declara la URL del listado (/testimonios), no cada comentario por separado, y ese listado está paginado (10 comentarios por página). El ejercicio es igual en espíritu al de clientes (c), pero aquí el foco está puesto en usar BeautifulSoup para **recorrer manualmente la paginación** hasta agotar todas las páginas de comentarios.

```
import csv
import os
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)

def hay_pagina_siguiente(soup: BeautifulSoup) -> bool:
    nav = soup.select_one('nav[aria-label="Paginacion"]')
    if nav is None:
        return False
    return any(enlace.get_text(strip=True) == "Siguiente" for enlace in
nav.select("a"))
```

1. Descubrir la URL del listado de testimonios a partir del sitemap

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
testimonios_sitemap_url = next(url for url in sub_sitemaps if "sitemap-testimonios"
in url)

sitemap_testimonios = get_soup(testimonios_sitemap_url, "xml")
testimonios_base_url = sitemap_testimonios.find("loc").text
print("Listado base de testimonios:", testimonios_base_url)
```

Listado base de testimonios: http://localhost:3000/testimonios

2. Scraping de cada comentario

Cada testimonio es un `<li data-comentario-id itemscope itemtype="https://schema.org/Review">` con atributos `data-*` para el cliente, la calificación y la fecha, y microdatos `itemprop` para el texto de la reseña.

Detalle a tener en cuenta: el nombre completo del cliente se renderiza como `{nombre} {apellidos}` dentro de un mismo elemento `itemprop="name"`, es decir, React separa ambos valores con un nodo de texto que es un único espacio. Si se usa `get_text(strip=True)` sin especificar un separador, BeautifulSoup termina eliminando ese espacio (porque `strip=True` vacía los nodos de texto que solo contienen espacios antes de unirlos), y el resultado queda pegado (por ejemplo `"UrsulaCastaneda"`). La forma correcta es pasar un separador explícito: `get_text(" ", strip=True)`.

```
def scrape_comentario(li) -> dict:
    autor = li.select_one('[itemprop=author]')
    nombre_completo = autor.select_one('[itemprop=name]').get_text(" ", strip=True)
    return {
        "id": li["data-comentario-id"],
        "cliente_id": li["data-cliente-id"],
        "cliente_nombre_completo": nombre_completo,
        "cliente_ciudad": li.select_one('[data-ciudad]')['data-ciudad'],
        "cliente_pais": li.select_one('[data-pais]')['data-pais'],
        "calificacion": li["data-calificacion"],
        "fecha": li["data-fecha"],
        "texto": li.select_one('[itemprop=reviewBody]').get_text(strip=True),
    }
```

3. Recorrer todas las páginas hasta que no haya un enlace "Siguiente"

```
comentarios = []
numero_pagina = 1

while True:
    soup = get_soup(f"{testimonios_base_url}?page={numero_pagina}")
    items = soup.select("li[data-comentario-id]")
    if not items:
        break

    comentarios.extend(scrape_comentario(item) for item in items)
    print(f"Pagina {numero_pagina}: {len(items)} comentarios")

    if not hay_pagina_siguiente(soup):
        break
    numero_pagina += 1

print(f"\nTotal de comentarios scrapeados: {len(comentarios)} en {numero_pagina} paginas")
```

Pagina 1: 10 comentarios
Pagina 2: 10 comentarios
Pagina 3: 10 comentarios
Pagina 4: 10 comentarios
Pagina 5: 10 comentarios
Pagina 6: 10 comentarios
Pagina 7: 10 comentarios
Pagina 8: 10 comentarios
Pagina 9: 10 comentarios
Pagina 10: 10 comentarios
Pagina 11: 10 comentarios
Pagina 12: 10 comentarios
Pagina 13: 10 comentarios
Pagina 14: 5 comentarios

Total de comentarios scrapeados: 135 en 14 paginas

4. Guardar los datos en data/comentarios.csv

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "comentarios.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=comentarios[0].keys())
    writer.writeheader()
    writer.writerows(comentarios)

print(f"Se guardaron {len(comentarios)} comentarios en {OUTPUT_PATH}")
```

Se guardaron 135 comentarios en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-02/notebooks/./data/comentarios.csv